Claude Code × Neo4j × MCP: 한국 자동차 산업지식그래프 실습

실습 프롬프트 중심 강의노트 | K-Auto KG v1.1

2026-10-19

Claude Code × Neo4j × MCP: 한국 자동차 산업지식그래프 구축

핵심 목표: 수강생이 '이미 완성된 그래프'를 보는 데 그치지 않고, **Claude Code에 무엇을 어떻게 지시해야 그래프가 생성·검증되는지** 재현한다. 본 교안은 1단계 **K-Auto KG**만 다룬다. 2단계 **NeoQuant 금융 온톨로지·GraphRAG**는 확장 로드맵으로만 소개한다.

전제: 본 강의는 제공된 KAuto_KnowledgeGraph_Stage1_v1.1_CompanyRelations.zip과 그 안의 kauto_kg_mvp/ 폴더를 사용한다. 수업 중 실행 환경마다 설치·연결 결과가 달라질 수 있다. '예상 결과'와 '실측 결과'를 혼동하지 않는다.

학습성과 및 제출물

실습을 마치면 수강생은 다음을 할 수 있다.

Claude Code에게 현재 PC 환경을 **조사→계획→실행→검증**하도록 지시한다.

Neo4j 그래프의 **노드, 관계, 속성, Cypher**를 설명하고 직접 질의한다.

기업→밸류체인, 기업→역량→차량 시스템, **기업↔기업 관계**를 구별해 적재한다.

실제 납품, **공급계약**, **기술협력**, **계열 관계**를 혼동하지 않고 출처를 추적한다.

정적 HTML 미리보기, Neo4j Browser, Claude Code MCP의 서로 다른 역할을 구분한다.

오류 발생 시 Claude Code에 원인 진단·재검증을 체계적으로 요구한다.

최종 제출: (a) 실행한 프롬프트 3개 이상, (b) 실행 로그의 성공/실패 판정, (c) Neo4j 그래프 또는 표 화면 2개, (d) 기업 간 관계 1건의 출처·상태 분석, (e) 본인이 수정한 질문·Cypher 1개를 student_report.md에 정리한다. 비밀번호·토큰·사내 비공개 자료는 제출하지 않는다.

0. 강의 운영표

구간	주제	핵심 입력	완료 확인	권장 시간
A	준비·역할 이해	준비 체크리스트, P0	OS·Docker·Python·Claude 확인	30분
B	설치·접속	P1~P3	Neo4j Browser 접속, 데이터 변경 전 승인	65분
C	데이터·온톨로지	P4~P5	CSV 6개 단위 테스트 PASS, 스키마 설명	

45분

D

적재·검증

P6~P7

데이터 30/7/24/42 확인

55분

E

기업 간 관계 탐색

P8~P10

공급계약과 납품 구분, 출처 열기

55분

F

MCP·응용·발표

P11~P14

MCP 읽기 질의·수정 프롬프트·결과 보고

55분

시간표는 수업 진행 가이드다. Docker Desktop 설치·첫 로그인은 환경에 따라 추가 시간이 필요하다. **이미 Neo4j가 설치되어 있다면 B 구간의 설치는 생략하고 연결만 검증**한다.

Part I. 실습에 들어가기 전에

1. 최소 개념: 무엇이 무엇을 하는가?

구성요소

한 문장 설명

실제 수강생 행동

Claude Code(클코)

자연어 지시를 받아 파일·명령어·프로그램 작업을 수행하는 개발 에이전트

프로젝트 폴더에서 claude 실행 후 프롬프트 입력

Docker Desktop

Neo4j DB를 격리된 컨테이너로 실행

설치·실행 여부 확인, 필요시 직접 관리자 승인

Neo4j

기업과 관계를 저장하는 그래프 데이터베이스

http://localhost:7474에서 그래프 확인

Cypher

Neo4j에 질문하는 언어

MATCH … RETURN … 실행

MCP

Claude Code가 DB 조회 도구를 사용할 수 있게 하는 연결 표준

claude mcp list, /mcp로 확인

Python Driver

CSV를 Neo4j에 적재하고 검증하는 코드 연결

python scripts/import_neo4j.py 등 실행

정적 HTML

CSV로 만든 그래프 미리보기. DB와 실시간 연결되지 않음

output/kauto_graph.html 열기

기본 구조는 **사용자 → Claude Code → (터미널·Python·MCP) → Neo4j** 이다. 별도로 **CSV → HTML 미리보기** 경로가 있다. HTML을 열었다고 해서 DB에 입력되었다는 뜻은 아니다.

꼭 구별해야 할 사실과 추론

HAS_CAPABILITY: 어느 기업이 특정 기술·제품 역량을 가진다고 분류한 연결.

ENABLES_SYSTEM: 어떤 역량이 차량 시스템에 연관된다는 산업 개념 연결.

AFFILIATED_WITH: 같은 기업집단에 속함. **지분 소유를 자동 의미하지 않는다**.

SUPPLY_AGREEMENT_WITH: 공급계약 발표. **실제 물품 납품 완료와 다르다**.

SUPPLIES_TO: 공식 근거에서 기존 공급·납품이 명시된 관계.

TECH_PARTNERS_WITH, JOINT_RESEARCH_WITH: 협력·공동연구. **독점·실적·납품을 자동 의미하지 않는다**.

CO_MEMBER_OF_CONSORTIUM: 동일 컨소시엄 참여. **직접 거래를 자동 의미하지 않는다**.

JOINT_VENTURE_PLAN_WITH: 합작사업 관련 발표. **설립 완료나 현재 운영 상태를 확정하지 않는다**.

2. 준비물: 각자 PC에서 확인할 것

필수: 인터넷 연결, Windows 11/macOS/Linux, 터미널(PowerShell 또는 zsh), Python 3, Claude Code와 로그인/사용 권한, 프로젝트 ZIP. **권장:** Docker Desktop(Windows는 WSL2 가상화 필요), VS Code, 최신 브라우저. Docker 설치가 제한된 회사 PC라면 강사에게 기존 Neo4j 또는 별도 실습 DB 제공을 요청한다.

소프트웨어 버전은 고정하지 말고 현재 설치 환경을 점검한다. 단, 제공 패키지의 Docker Compose는 개발 실습의 재현성을 위해 neo4j:2026.09.0을 지정한다. 이미 Neo4j가 있다면 임의로 교체하지 않는다.

수강생 준비 체크리스트

☐ KAuto_KnowledgeGraph_Stage1_v1.1_CompanyRelations.zip 다운로드 및 압축 해제

□ 압축 해제 후 kauto_kg_mvp/README.md와 CLAUDE_SETUP_PROMPT.md 파일 존재 확인

□ 터미널을 **반드시 kauto_kg_mvp/ 폴더 안에서** 실행

□ Claude Code 로그인 확인. claude --version 실행 가능

□ Python 확인: Windows py --version, macOS python3 --version

□ Docker 확인: docker --version 및 docker compose version; Desktop 실제 실행 여부 별도 확인

□ 비밀번호는 공개 채팅·공유 문서·Git에 적지 않기

□ 개인 PC가 아닌 경우 로컬 포트 7474·7687 사용과 소프트웨어 설치 권한 확인

Windows: 폴더를 여는 정확한 방법

압축 해제한 kauto_kg_mvp 폴더를 파일 탐색기에서 연다.

주소 표시줄에 powershell을 입력하고 Enter를 누른다. **현재 폴더가 명령 실행 위치**가 된다.

Get-Location을 입력해 마지막 폴더명이 kauto_kg_mvp인지 확인한다.

claude를 실행한다. 명령이 없으면 Claude Code가 아직 설치되지 않았거나 PATH가 연결되지 않은 것이므로 P1에서 설치 지원을 요청한다.

macOS: 폴더를 여는 정확한 방법

Finder에서 압축 해제한 kauto_kg_mvp 폴더를 준비한다.

Terminal을 열고 cd 뒤에 해당 폴더를 끌어다 놓아 Enter를 누른다.

pwd로 경로를 확인한 후 claude를 실행한다.

강사용 원칙

한 번에 파괴적 명령 전체를 승인하지 않는다. 도커 볼륨 삭제, MATCH (n) DETACH DELETE n, DB 초기화, 전역 MCP 비밀번호 저장은 실습에서 금지한다. 작성·수정·삭제가 발생하는 부분은 먼저 설명을 요구하고 승인한다.

Part II. 따라 하기: 정확한 프롬프트와 명령어

각 실습은 **어디 → 입력 → 예상 출력 → 확인 → 실패 시 다음 지시** 순서로 구성했다. P0~P14는 Claude Code 채팅에 붙여 넣을 프롬프트, T는 터미널 명령, C는 Neo4j Browser의 Cypher다.

실습 A | 환경 진단 — P0, P1

목표: 무조건 설치하지 않고 현재 PC 상태를 확인한다.

어디: kauto_kg_mvp/에서 실행한 Claude Code 대화창.

P0 — 현황 조사(실행·수정 없이 먼저 진단)

```
너는 초보자를 도와주는 환경구축 코치다.
현재 폴더의 README.md, CLAUDE.md, CLAUDE_SETUP_PROMPT.md를 읽고
운영체제, 셸, Python, Claude Code, Docker/Compose, 7474·7687 포트,
기존 Neo4j 및 MCP 등록 상태를 확인해줘.
지금은 설치하거나 설정을 바꾸지 말고,
(1) 이미 준비된 것 (2) 미설치 (3) 확인 불가를 표로 보고해줘.
다음 행동은 하나씩, '어디 → 입력 → 출력 → 확인'으로 설명해줘.
```

예상 출력: OS/셸별 설치·미설치 표, 다음 작업 제안. **확인:** 실제 탐지 결과가 출력되었는가? 근거 없는 '설치됨' 표시는 없는가?

P1 — 설치 계획과 승인 경계

```
P0 진단 결과를 기준으로 이미 있는 프로그램은 재설치하지 마.
필요할 때만 공식 설치 경로를 사용해 Docker Desktop, Python, Claude Code를
준비할 계획을 세워줘. 관리자 승인·로그인·비밀번호 입력은 내게 안내하고
직접 승인이 필요한 작업 직전에 멈춰줘. 기존 DB와 도커 볼륨을 삭제하지 마.
설치 전후의 버전 명령으로 성공을 재확인해줘.
```

수동 확인용 T1(터미널)

```
# Windows PowerShell
Get-Location
py --version
claude --version
docker --version
docker compose version
claude mcp list

# macOS Terminal
pwd
python3 --version
claude --version
docker --version
docker compose version
claude mcp list
```

docker --version만 성공하고 docker compose ps가 실패한다면 **Docker CLI는 있지만 Docker Desktop 엔진이 실행 중이 아닐 수 있다.** P1에게 차이를 설명하도록 한다.

실습 B | 로컬 Neo4j 시작 — P2, P3

목표: 안전한 로컬 DB를 실행하고 Browser에 로그인한다.

P2 — 비밀정보 설정 및 기존 DB 재사용 점검

```
.env.example과 docker-compose.yml을 읽고,
설정에 필요한 NEO4J_URI, NEO4J_USERNAME, NEO4J_DATABASE가 무엇인지 알려줘.
.env가 없으면 .env.example을 복사하되, 비밀번호는 내 로컬 편집기에서
직접 설정하게 안내해줘. 비밀번호 값은 출력하거나 Git에 추가하지 마.
기존에 Neo4j가 7474/7687 포트를 사용하면 새 DB를 띄우지 말고
재사용 또는 포트 변경 방안을 먼저 설명해줘.
```

수동 T2 — .env 만들기(기존 파일을 함부로 덮어쓰지 않는다)

```
# Windows PowerShell, .env가 없을 때만
Copy-Item .env.example .env
notepad .env

# macOS Terminal, .env가 없을 때만
cp .env.example .env
open -e .env
```

.env의 NEO4J_PASSWORD=CHANGE_TO_A_LONG_UNIQUE_PASSWORD 부분을 **개인용 긴 비밀번호로 바꾼다.** 한글 안내문이 아니라 실제 값으로 입력한다. 이미 생성된 Docker 볼륨의 DB 비밀번호는 .env를 바꾸는 것만으로 자동 변경되지 않는다.

P3 — 컨테이너 시작(검증 위주)

```
기존 Neo4j 인스턴스와 데이터 볼륨을 먼저 확인해줘.
새 실습 DB가 필요한 경우에만 docker compose up -d를 실행하고,
컨테이너 상태와 로컬 포트 응답을 확인해줘.
Neo4j Browser(http://localhost:7474)에서 어느 값으로 로그인하는지
주소, 사용자명, 비밀번호를 어디서 읽어야 하는지만 알려줘.
비밀번호 값은 그대로 가려줘. 기존 볼륨은 절대 삭제하지 마.
```

수동 T3

```
docker compose up -d
docker compose ps
```

예상 출력: 컨테이너 kauto-neo4j 상태 실행 중, 브라우저에서 로그인 가능. **확인:** http://localhost:7474 접속→로그인→RETURN 1 AS ok; 실행 시 표에 1이 표시되는가?

```
RETURN 1 AS ok;
```

주의: 이 시점에는 아직 기업 데이터가 없을 수 있다. Browser가 열려도 적재 성공은 아니다.

실습 C | Python과 데이터 테스트 — P4

P4 — Python 가상환경 설치·테스트 요청

```
현재 프로젝트 폴더에서 Python 가상환경(.venv)을 만들거나 재사용해줘.
requirements-mcp.txt를 사용해 필요한 패키지를 설치하고
패키지 설치 성공 여부를 확인해줘.
이어서 python -m unittest discover -s tests -v를 실행해
6개 데이터 무결성 검사의 실제 PASS/FAIL 목록을 보여줘.
실패하면 원인·수정안·재실행 로그를 함께 보고해줘.
```

직접 실행 T4 — Windows PowerShell

```
py -m venv .venv
.venv\Scripts\python.exe -m pip install -r requirements-mcp.txt
.venv\Scripts\python.exe -m unittest discover -s tests -v
```

직접 실행 T4 — macOS/Linux

```
python3 -m venv .venv
.venv/bin/python -m pip install -r requirements-mcp.txt
.venv/bin/python -m unittest discover -s tests -v
```

예상 출력: 테스트 6건 ok, 마지막 Ran 6 tests ... OK. 실제 데이터 검사는 패키지에 포함된 CSV를 검증할 뿐 Neo4j 서버 연결을 보증하지 않는다.

실습 D | 온톨로지 이해 및 프롬프트로 설계 설명 — P5

핵심 파일: data/companies.csv, data/company_relationships.csv, data/capabilities.csv, data/stages.csv, data/stage_links.csv.

P5 — 스키마 설계 의도 읽기

이 프로젝트의 CSV와 scripts/import_neo4j.py를 읽고
한국 자동차 산업 온톨로지를 초보자에게 설명해줘.
노드 타입 6개 이상과 주요 관계 타입을 구분하고,
각각의 실제 CSV 열에 매핑해줘.
특히 기업→역량→차량 시스템과 기업↔기업 간 관계의 차이를
현대모비스, 포스코퓨처엠 사례로 비교해줘.
확인된 사실·산업분류·추정을 다른 표로 구분해줘.
모호한 관계는 새로 만들어내지 마.

예상 출력: 엔티티·관계·속성 매핑표. 확인할 주요 레이블은 KAutoCompany, KAutoStage, KAutoCapability, KAutoSystem, KAutoSource, KAutoRelationAssertion이다.

실습생 연습 질문: "같은 그룹에 속한 두 기업이 반드시 직접 소유 관계인가?" → **아니다**. "전기차 관련 역량이 같으면 납품관계가 있는가?" → **아니다**.

필수 관계 데이터 모형

company_relationships.csv의 주요 열:

id, from_company_id, to_company_id, relation_type,
relation_label_ko, topic, announcement_date, assertion_status,
evidence_url, evidence_summary_ko, evidence_level, reviewed_on

announcement_date는 보도·발표일이다. 사업 현재 상태를 보증하지 않는다. reviewed_on은 자료 검토일이다. evidence_url은 관계의 주장에 대한 근거이며, 기업 대표 홈페이지 주소와 다를 수 있다.

실습 E | 데이터 적재와 DB 검증 — P6, P7

데이터베이스에 실제 노드·관계를 쓰는 단계다. **실습 전 승인**이 반드시 필요하다.

P6 — 적재 이전 안전 검토 및 승인 후 실행

데이터 적재 작업을 준비해줘.
(1) DB 주소/이름과 기존 데이터 존재 여부 (2) 새로 추가·수정할 노드·관계
(3) 백업 필요 여부 (4) MERGE 중복방지 로직을 먼저 설명해줘.
비밀번호나 연결 비밀을 출력하지 마.
운영 DB와 테스트 DB가 구별되면 테스트 DB를 사용해줘.
실제 쓰기 작업 직전에 내 승인을 받고, 승인 후에만
scripts/import_neo4j.py를 실행해줘. 기존 데이터를 삭제하지 마.

수동 T5(승인 후 가상환경 Python으로 실행)

```
# Windows
.venv\Scripts\python.exe scripts/import_neo4j.py
.venv\Scripts\python.exe scripts/verify_neo4j.py

# macOS/Linux
.venv/bin/python scripts/import_neo4j.py
.venv/bin/python scripts/verify_neo4j.py
```

P7 — 검증 결과 표 만들기

```
scripts/verify_neo4j.py를 실제로 실행하고 다음을 표로 보고해줘:
기업, 밸류체인 단계, 핵심역량, 차량시스템,
기업 간 관계, 관계 주장(Assertion), 출처 노드,
출처·상태를 가진 관계 수, 금지된 범용 SUPPLIES 관계 수.
기대치와 실제치를 별도 열로 두고 PASS/FAIL을 표시해줘.
실행하지 못했다면 UNVERIFIED로 남겨줘.
```

시드 데이터의 기준값: 기업 30, 단계 7, 핵심역량 24, 기업↔기업 관계 42, 관계 주장 42, SUPPLIES 0. 시스템·출처 수는 verify_neo4j.py의 실측값으로 확인한다. 이 수치는 제공 패키지 시점의 **교육용 데이터셋 기준값**이며 한국 산업 전체의 기업·계약 총수가 아니다.

참고 Cypher C1 — 기업과 관계 수 확인

```
MATCH (c:KAutoCompany) RETURN count(c) AS company_count;

MATCH (:KAutoCompany)-[r]->(:KAutoCompany)
WHERE r.project='kauto-stage1'
RETURN type(r) AS relation_type, count(*) AS relation_count
ORDER BY relation_count DESC;
```

참고 Cypher C2 — 관계와 출처 함께 확인

```
MATCH (a:KAutoCompany)-[r]->(b:KAutoCompany)
WHERE r.project='kauto-stage1'
RETURN a.name_ko AS 출발기업, type(r) AS 관계, b.name_ko AS 도착기업,
       r.topic AS 주제, r.assertion_status AS 상태,
       r.evidence_url AS 근거URL
ORDER BY 관계, 출발기업 LIMIT 50;
```

실습 F | DB가 없어도 되는 미리보기 — P8

P8 — HTML 생성과 차이 설명

```
scripts/render_html.py를 실행해 output/kauto_graph.html을 만들고,
파일을 브라우저에서 여는 방법을 내 OS 기준으로 안내해줘.
기업을 선택하고 기업↔기업 관계유형 필터를 바꿀 때
어떤 노드와 선이 나타나는지 설명해줘.
반드시 이것이 CSV 기반 '정적 미리보기'이며 Neo4j 실시간 조회가
아니라는 점을 화면에 표시하고 보고해줘.
```

수동 T6

```
# Windows
.venv\Scripts\python.exe scripts/render_html.py
Start-Process .\output\kauto_graph.html

# macOS
.venv/bin/python scripts/render_html.py
open output/kauto_graph.html
```

확인: 기업 선택, 관계유형 필터, 공식 출처 링크가 작동하는가? **유의:** 수업에서는 HTML의 클릭 경험과 Neo4j Browser의 실DB 질의를 명확히 구분한다.

실습 G | 네 가지 질문을 프롬프트에서 Cypher로 — P9

어디: Claude Code에 프롬프트를 입력하고, 생성된 Cypher를 Neo4j Browser의 상단 입력창에 붙여 넣는다. 또는 패키지 queries/*.cypher 파일에서 이미 준비된 검증용 쿼리를 사용한다.

P9 — 질문→Cypher를 만들게 하기

읽기 전용 Cypher만 사용해 한국 자동차 산업지도를 탐색하고 싶어.
다음 4개 질문에 대해 각각 (1) Cypher (2) 기대되는 그래프/표 형태
(3) 결과가 의미하지 않는 것 (4) 출처 확인 방법을 작성해줘.
A. 밸류체인별 기업 수
B. 현대모비스→핵심역량→차량 시스템
C. 현대자동차와 직접 연결된 기업·관계유형
D. 포스코퓨처엠 관련 공급계약과 실제 납품의 차이
프로젝트 queries/01, 02, 07, 08 파일을 먼저 읽고
필요하면 실제 스키마 이름에 맞춰 수정해줘.

비교용 쿼리

```
// C3 기업별 밸류체인
MATCH (c:KAutoCompany)-[:PARTICIPATES_IN_STAGE]->(s:KAutoStage)
RETURN s.name_ko AS 밸류체인, count(c) AS 기업수, collect(c.name_ko) AS 기업
ORDER BY 기업수 DESC;

// C4 현대모비스 역량 그래프: 그래프 보기에서 실행
MATCH (c:KAutoCompany {id:'company.mobis'})-[r:HAS_CAPABILITY]->(p:KAutoCapability)
      -[e:ENABLES_SYSTEM]->(s:KAutoSystem)
RETURN c,r,p,e,s;

// C5 현대자동차 관련 직접관계 그래프
MATCH p=(c:KAutoCompany {id:'company.hyundai'})-[r]-(other:KAutoCompany)
WHERE r.project='kauto-stage1'
RETURN p LIMIT 50;

// C6 실제 납품 vs 공급계약: 단순 종류 수
MATCH (:KAutoCompany)-[r:SUPPLIES_TO|SUPPLY_AGREEMENT_WITH]->(:KAutoCompany)
RETURN type(r) AS 관계유형, count(*) AS 수;
```

초기 데이터의 예상 구분: SUPPLIES_TO 1건 / SUPPLY_AGREEMENT_WITH 3건. 다만 이는 **해당 시드의 출처 기록 수**다. 산업 전체의 납품·계약 수와는 다르다.

실습 H | '기업 간 관계'를 제대로 분석하기 — P10

P10 — 기업 간 관계의 검증 가능성 평가

company_relationships.csv를 기준으로
현대자동차·현대모비스·포스코퓨처엠 중 두 기업을 골라
서로 연결된 공개자료 기반 관계를 탐색해줘.
각 결과를 (A) 관계 타입, (B) 방향, (C) 주제,
(D) 발표일, (E) 진척상태, (F) 1차 출처 URL로 정리해줘.
기업이 같은 그룹이라는 사실에서 소유비율을 추정하지 말고,
공급계약 발표에서 실제 납품량·매출액을 추정하지 마.
증거가 부족하면 '미확인'으로 표시해줘.

읽어볼 경로: queries/07_company_network.cypher, queries/08_supplier_network.cypher, queries/09_company_risk_paths.cypher.

핵심 토론: 포스코퓨처엠에서 2홉 그래프상 연결되는 기업이 있다고 해서 그 기업의 생산 차질이 자동으로 다른 기업의 매출액에 전파된다고 말할 수는 없다. 2단계 NeoQuant에서 매출의존도·지분비율·대체조달성 등 정량 속성이 추가되어야 한다.

C7 — 연결별 근거 공개

```
MATCH (a:KAutoCompany)-[r]->(b:KAutoCompany)
WHERE r.project='kauto-stage1' AND r.evidence_url IS NOT NULL
RETURN a.name_ko AS A, type(r) AS 관계, b.name_ko AS B,
       r.announcement_date AS 발표일, r.assertion_status AS 상태,
       r.evidence_url AS 근거
LIMIT 20;
```

실습 I | Claude Code와 Neo4j MCP 연결 — P11, P12

목표: 터미널에서 Python으로 적재하는 것과 별개로, Claude Code가 **MCP를 통해 읽기 조회**를 수행하게 한다.

P11 — MCP 등록·비밀정보 보호

```
scripts/mcp_launcher.py, requirements-mcp.txt, README.md를 읽고
Neo4j MCP 서버를 현재 프로젝트에서 사용하도록 연결해줘.
먼저 같은 이름의 MCP가 등록되어 있는지 확인하고 중복 등록하지 마.
서버는 read-only여야 하고, 비밀번호는 .env에서만 읽어야 해.
현재 OS에 맞는 .venv Python과 mcp_launcher.py의 '절대 경로'로
claude mcp add 명령을 작성·실행하고 등록 결과를 확인해줘.
명령줄·설정 파일·로그에 비밀번호를 그대로 노출시키지 마.
```

명령 T7 — 정확한 경로는 각 PC마다 다름

```
claude mcp add neo4j-kauto -- <가상환경_Python의_절대경로> <scripts/mcp_launcher.py의_절대경로>
claude mcp list
```

<...>는 **그대로 복사해 실행하는 코드가 아니다.** P11에게 실제 절대 경로를 생성하도록 맡긴다. MCP를 등록한 뒤 현재 Claude Code 세션을 다시 시작해야 연결이 인식될 수도 있다. 권한 범위·옵션은 현재 Claude Code 버전에 맞춰 공식 문서를 확인한다.

P12 — 읽기 전용 MCP로 질문하기

```
등록된 neo4j-kauto MCP 도구를 사용해서만 현재 DB를 읽어줘.
(1) 노드 레이블 목록 (2) 기업 노드 개수
(3) 기업↔기업 관계 타입별 개수
(4) 현대모비스 연결 역량 5개 이내를 조회해줘.
사용한 도구명과 실행 결과를 보여줘.
MCP 연결 실패 시 로컬 Python으로 대신 조회하고도 'MCP 성공'이라고
표시하지 말고, 실패 원인과 해결 단계를 알려줘.
절대 데이터 쓰기 명령을 실행하지 마.
```

성공 판정: claude mcp list에 등록됨 **AND** Claude Code 세션에서 실제 MCP 도구 조회 결과가 돌아옴. 등록 목록에 보이기만 하는 것은 충분하지 않다.

실습 J | 결과 개선: 반복 프롬프트 — P13, P14

P13 — 설명가능성·감사기록 강화

지금 생성한 기업 간 그래프의 어떤 관계든 클릭하거나 조회하면
(1) 연결 기업 (2) relation_type (3) 주제 (4) 발표일
(5) evidence_url (6) 현재상태 확인 여부를 볼 수 있게 개선안을 제시해줘.
HTML 화면과 Neo4j Cypher 중 어느 부분을 수정할지 먼저 설계하고
수정 전후 기능 목록과 테스트 계획을 보여줘.
출처 없는 신규 거래관계는 만들지 마.
파일 수정 전에 변경되는 파일과 검증방법을 알려줘.

P14 — 추가 1개 기업/관계의 증거기반 보강(선택 실습)

신규 기업 1곳 또는 기존 기업 사이 관계 1건을 추가하는 연습을 하고 싶어.
공식 1차 출처부터 찾고, 관계 타입이 허용 목록에 있는지 검사해줘.
발표일·진척상태·원문 핵심 문장·출처 URL을 별도 표로 정리하고,
CSV 수정 제안(diff)을 먼저 보여줘. 내가 승인하기 전에는 저장하지 마.
승인 후 데이터 단위 테스트, HTML 재생성, DB MERGE 적재와
read-only 조회를 순서대로 검증해줘.
증거 없는 항목은 추가하지 마.

Part III. 실습을 잘하게 만드는 프롬프트 설계법

3. 재사용 가능한 프롬프트 틀

[역할] 너는 {역할} 전문가이자 초보자 실습 코치다.
[목표] {이번 시간에 구현할 한 가지 결과}를 달성해줘.
[자료] 반드시 먼저 {파일명/폴더명}을 읽어줘.
[제약] {기존 데이터 삭제 금지, read-only, 증거 없는 관계 금지}.
[실행] ① 현황 점검 ② 계획 ③ 실행 ④ 테스트 ⑤ 최종 보고.
[승인] 실제 DB 쓰기/계정 설치/외부 공개 전에 내 승인을 받아.
[보고] 어디 → 입력 → 예상 출력 → 실제 출력 → 확인 기준.
[실패] 오류를 숨기지 말고 원인과 다음 검증 명령을 제시해줘.

부정확한 요청: “한국 자동차 산업지도를 그려줘.” → 데이터·출처·관계 정의가 불명확하다.

개선한 요청: “제공 CSV의 기업 30개를 기준으로 공식 출처가 있는 관계만 온톨로지에 적재하고, 단위 테스트 6건과 기업 간 관계 42건을 검증한 뒤, Neo4j Browser에서 탐색할 Cypher 3개를 생성해줘. 추정 납품은 금지.” → 범위·입력·출처·금지조건·검증이 있다.

4. ‘출력’ 대신 ’생성 과정’을 평가하기

평가 질문

좋은 작업의 증거

부족한 상태

무엇을 읽었나?

README, CSV 스키마, 쿼리 경로 언급

출처 없는 일반 설명

무엇을 실행했나?

정확한 명령과 종료코드·로그

‘완료했습니다’ 한 문장

어디에 결과가 생겼나?

HTML/DB 이름과 검증 경로

그림만 첨부

무엇을 사실로 취급하나?

계약·납품·협력 구분

그래프의 모든 선을 거래로 해석

실패를 어떻게 처리했나?

원인→수정→재시험

오류 무시

재현 가능한가?

다른 PC에서 동일 파일·명령으로 재실행 가능

개인 환경에만 의존

5. 막혔을 때 사용하는 진단 프롬프트

D1: claude 명령을 찾을 수 없음

'claude' 명령을 찾을 수 없다는 오류가 났어.
운영체제, 설치 여부, PATH 문제를 구분하고 공식 설치 경로만 알려줘.
확인 명령부터 하나씩 진행하고 이전에 설치된 것은 보존해줘.

D2: Docker daemon 연결 실패

docker compose ps 실행이 실패했어. Docker CLI 설치 여부와
Docker Desktop 실제 실행 여부를 분리해서 진단해줘.
Windows라면 WSL2/가상화 상태도 확인하고 GUI에서 클릭할 위치를 알려줘.
기존 컨테이너와 볼륨은 수정하지 마.

D3: Neo4j 인증 실패

Neo4j에서 Authentication failed가 발생했어.
.env 설정, 사용자 이름, 기존 Docker 데이터 볼륨의 비밀번호가
서로 다른 경우를 우선 점검해줘. 비밀번호는 표시하지 마.
기존 데이터 삭제나 비밀번호 초기화는 하지 말고 안전한 복구 경로를 알려줘.

D4: ModuleNotFoundError

현재 프로젝트의 Python 인터프리터와 pip 설치 인터프리터가 같은지
경로로 확인해줘. requirements-mcp.txt 설치 상태를 점검하고
가상환경 Python을 직접 호출하도록 명령을 수정해줘.

D5: MCP는 목록에 있지만 호출이 안 됨

claude mcp list에는 neo4j-kauto가 있지만 Claude Code가 도구를
호출하지 못해. 등록된 실행기의 절대 경로, 패키지 설치, .env 위치,
read-only 설정, 서버 프로세스 오류를 순서대로 진단해줘.
민감값은 마스킹하고 /mcp에서 실제 질의가 성공하는지 검증해줘.

D6: 42건 대신 다른 수가 나옴

기업간 관계 수가 42가 아니야. 제공 CSV 건수, DB명, 중복 ID,
관계 타입·project 속성, 이전 적재 여부를 읽기 쿼리로 검사해줘.
임의 삭제나 전체 DB 초기화는 하지 말고 차이 목록과 수정 전 계획을 제시해줘.

Part IV. 심화 과제와 평가

과제 A — 실제 관계를 잘못 해석한 설명 고치기

학생에게 다음 문장을 제시한다. “포스코퓨처엠과 배터리 회사가 연결되어 있으니 현재 실제로 모든 계약 물량을 납품하고 있다.” 오류의 위치를 지적하고, **증거 유형·계약 체결·공급이행**을 구분해 5줄 이내로 수정한다.

과제 B — 2홉 경로의 의미 파악

queries/09_company_risk_paths.cypher를 실행하고 연결 경로 하나를 택한다. 각 관계의 타입·증거와 경로가 **암시하지 않는 것(신용리스크 전파량, 공급액, 의존도)**을 함께 보고한다.

과제 C — 한 가지 조회 질문을 스스로 만들기

예: “어떤 기업들은 같은 차량 시스템에 연결되어 있지만 공식 납품관계는 검증되지 않았는가?” Claude Code에 질의 설계 요청 → Cypher 초안 → Browser 실행 → 오류 수정 → 결과 해석까지 전 과정을 기록한다.

권장 평가표(100점)

항목	배점	채점 기준
재현 가능성	25	어디서 어떤 명령을 실행했는지 분명
프롬프트 품질	25	

역할·파일·제약·승인·검증·실패 처리가 명시됨

그래프·Cypher

20

2개 이상 결과 재현 및 쿼리 수정 1회

증거·관계 구별

20

납품/계약/협력/계열을 정확히 구분

오류 대응·발표

10

실패 로그 또는 확인 절차 정리

보너스: 오류를 실제로 만나도 체계적으로 해결한 경우 긍정적으로 평가한다. '실패가 없었다'보다 '왜 성공했는지 증명할 수 있다'가 중요하다.

제출 템플릿 (student_report.md)

```
# K-Auto KG 실습보고서
- 실습일 / 운영체제:
- 사용 프로젝트 버전: v1.1
- 내 목표 질문:
## 내가 사용한 프롬프트
1. P__ : ...
2. P__ : ...
3. 나의 개선 프롬프트: ...
## 실제 실행 명령과 결과
- 명령 / 실행 위치 / PASS·FAIL / 확인한 로그:
## 내 그래프·표 2개
- 스크린샷·파일 위치 / 사용한 Cypher:
## 선택한 기업 간 관계 1건
- 기업 A → 관계 타입 → 기업 B:
- 발표일 / 상태 / 공식 URL:
- 확인된 사실:
- 아직 증명되지 않은 사실:
## 발생한 문제와 해결 방법
## 내가 새로 알게 된 점(3문장)
```

수업 종료 체크리스트

□ Python 단위 테스트 6개 PASS

□ CSV 미리보기 HTML 열기·필터 체험

□ Neo4j Browser에서 실제 DB 연결 확인(실행 환경 허용 시)

□ 기업 30, 단계 7, 역량 24, 기업간 관계 42 **실측값** 확인

□ Cypher 3종(역량·기업관계·공급계약/납품) 실행

□ MCP 등록 목록 확인 + **실제 읽기 질의 성공**(가능한 환경)

□ 선택한 관계 1건의 출처 및 진척상태 확인

□ student_report.md 작성, 비밀정보 삭제 확인

Part V. 강사용 운영 메모 / 2단계 연결

6. 강사의 실습 순서 제안

시작 5분은 완성된 HTML 미리보기를 보여주되 **직후에 이를 만들어내는 명령어(P8)**를 제시한다. 나머지는 P0→P14 흐름을 유지한다. 수강생이 서로 다른 OS를 사용할 때 설치 화면을 동시에 따라 하게 하지 말고, 일단 데이터 테스트 및 정적 HTML부터 성공하게 하는 **Plan B**를 준비한다.

Plan A: 설치 가능

Plan B: Docker 설치 제약

Docker + Neo4j + Python + MCP 완전 실습

Python 데이터 테스트 + 정적 HTML + Cypher 읽기/설계 연습

실제 DB 적재, Browser 그래프, MCP 조회 검증

DB 쿼리는 강사 시연 또는 공유 테스트 DB에서 확인

데이터 수 수정·비교까지 가능

CSV 증거검토·시각화·프롬프트 개선에 집중

7. 2단계 NeoQuant/GraphRAG로 확장할 때 재사용할 것(이번 시간 구현 대상 아님)

기업명·산업과 **유일 ID 및 관계 증거** 구조

KAutoSource, KAutoRelationAssertion을 활용한 **출처·추적성(provenance)** 설계

기업 간 단순 연결과 **정량 위험 노출(Exposure)**의 구분

데이터 검증, MERGE 적재, 읽기 전용 조회·권한 분리

이후 확장 객체: Security, Portfolio, Factor, RiskEvent, Paper, Strategy, Backtest, DocumentChunk

주의: GraphRAG, 기업별 매출 의존도, 공급망 위험 전파의 계량화는 1단계 수업 완료 기준이 아니다.

8. 명령어 한 페이지 치트 시트

목적

Windows PowerShell

macOS/Linux

폴더 위치 확인

Get-Location

pwd

Claude Code 시작

claude

claude

Docker 확인

docker compose ps

docker compose ps

DB 시작(필요할 때)

docker compose up -d

동일

가상환경 생성

py -m venv .venv

python3 -m venv .venv

패키지 설치

.venv\Scripts\python.exe -m pip install -r requirements-mcp.txt

.venv/bin/python -m pip install -r requirements-mcp.txt

단위 테스트

.venv\Scripts\python.exe -m unittest discover -s tests -v

.venv/bin/python -m unittest discover -s tests -v

HTML 생성

.venv\Scripts\python.exe scripts/render_html.py

.venv/bin/python scripts/render_html.py

DB 적재(승인 후)

.venv\Scripts\python.exe scripts/import_neo4j.py

.venv/bin/python scripts/import_neo4j.py

DB 검증

.venv\Scripts\python.exe scripts/verify_neo4j.py

.venv/bin/python scripts/verify_neo4j.py

MCP 등록확인

claude mcp list

동일

DB Browser

http://localhost:7474

동일

수업용 메인 프롬프트: 모든 실습을 한 번에 오케스트레이션

```
프로젝트의 README.md, CLAUDE_SETUP_PROMPT.md, CLAUDE.md,
RELATIONSHIP_MODEL_KR.md, EXERCISES_KR.md를 먼저 읽어줘.
나는 초보 수강생이다. K-Auto KG v1.1을 이 PC에서 재현하고 싶다.
단계를 (1) 환경진단 (2) 비밀정보 설정 (3) Neo4j 실행
(4) Python 설치·CSV 테스트 (5) 정적 HTML (6) 온톨로지 설명
(7) 사용자 승인 후 DB 적재 (8) Cypher 조회 (9) read-only MCP 연결
(10) 완료 체크리스트로 나눠줘.
각 단계에서 '어디 → 입력 → 실제 출력 → PASS/FAIL ➔ 다음 단계' 순서로
보고하고, 사용자가 직접 해야 하는 일은 한 번에 하나씩 안내해줘.
비밀번호는 출력하지 말고, 기존 DB·볼륨·파일 삭제는 금지해.
기업과 기업의 관계는 근거가 있는 42건의 시드만 사용하고,
실제 납품과 공급계약, 계열과 소유관계를 반드시 구분해.
실행하지 못한 것은 완료라고 표시하지 마.
완료 후 student_report.md의 초안을 만들어줘.
```

참고문서 및 정확성 고지

Neo4j MCP 설치: https://neo4j.com/docs/mcp/current/installation/

Neo4j MCP Claude Code 연결: https://neo4j.com/docs/mcp/current/client-configuration/

Neo4j Docker APOC: https://neo4j.com/docs/operations-manual/current/docker/plugins/

Neo4j Python Driver: https://neo4j.com/docs/python-manual/current/

Claude Code MCP 사용: https://code.claude.com/docs/en/mcp

실습 프로젝트: 제공 ZIP 내 README.md, CLAUDE_SETUP_PROMPT.md, RELATIONSHIP_MODEL_KR.md, queries/*.cypher

검증 범위(2026-10-19 기준): 교육용 패키지의 CSV 단위 테스트는 6개 통과. 이 강의자료 제작 환경에서 수강생 PC의 Docker·Neo4j 서버 실행, 데이터베이스 적재, Claude Code MCP 실제 연결 성공 여부는 검증할 수 없다. 기업 수·관계 수는 **시드 데이터셋의 기준값**이다. 개별 공급·협력 관계는 발표 시점과 현재 상태를 별도로 확인해야 한다.